# University Homework LaTeX Template

A General-Purpose Bilingual Template for Academic Assignments

University of Tehran

## 1 Introduction

This repository provides a versatile, general-purpose **academic homework template** designed for university courses across computer science, engineering, and mathematics. Originally adapted for courses at the **University of Tehran**, it is fully course-agnostic and can be effortlessly customized for any homework assignment, quiz, project description, or lab report.

## 2 Key Features

- **Course-Agnostic Design:** Easily configure the template for any course (e.g., Algorithms, Automata, Operating Systems, Machine Learning, Circuits) simply by modifying metadata variables.
- **Official Academic Header:** Built-in `\makeheader` generates a clean header with faculty logos, course title, semester, deadline, and instructor/TA contact info.
- **Bilingual & Persian Typesetting (XePersian):** Pre-configured with standard Persian fonts (*XB Niloofar*, *XB Zar*, *IRLotus*) and seamless bidirectional switching via `\LRE{}`.
- **Modular Subfiles Structure:** Manage large assignments by writing individual questions in standalone files (`Q1.tex`, `Q2.tex`) combined via `\subfile{...}`.
- **Conditional Solutions:** Toggle whether student solutions are compiled or omitted using the `compileanswers` boolean switch.
- **Syntax-Highlighted Code:** Formats source code elegantly using the `minted` package for Python, C/C++, Verilog, Java, and more.

## 3 Repository Structure

- `homework.cls` — Core document class. Originally by jez, enhanced by Behzad Shayegh, Borna Tavasoli, and Hesam Asadollahzadeh for Persian support. Adapted by Mehdi Jamalkhah and updated by Mohammad Khoshroo for TeX Live compatibility.
- `Background/` — University of Tehran and Faculty of Engineering logos.
- `Fonts/` — High-quality Persian TrueType fonts bundled directly in the repository.
- `HW.tex` — Master document defining course variables and including question subfiles.
- `latexmkrc` — Automated configuration for Overleaf and local `latexmk` builds.
- `LaTeX-build/` — Standard build directory for compiler artifacts (`.pdf`, `.log`, `.aux`).

## 4 Document Class Options

Pass these optional flags to `\documentclass[...]homework`:

- `largemargins` — Widens page margins to leave ample space for grading annotations.
- `newpage` — Automatically begins each new question on a fresh page.

- `anonymous` — Omits personal contributor details for blind grading.

# 5 Configuring for Any Course

To use this template for your course, define the metadata commands in your master `.tex` file:

```
\documentclass[12pt,largemargins]{homework}

\newcommand{\eventclass}{Design and Analysis of Algorithms} % Course name
\newcommand{\eventsemester}{Fall 2026}                      % Semester
\newcommand{\eventtype}{Homework}                           % Assignment type
\newcommand{\eventnum}{1}                                   % Assignment number
\newcommand{\eventcntrbs}{Instructor / TA Name}             % Author or TA
\newcommand{\hwemail}{ta-email@ut.ac.ir}                    % Contact email
\newcommand{\eventdatetype}{Due Date:}                      % Due label
\newcommand{\eventDeadlineDay}{25}                          % Day
\newcommand{\eventDeadlineMonth}{November}                  % Month
\newcommand{\eventDeadlineYear}{2026}                       % Year

\begin{document}
\makeheader
\subfile{./Q1.tex}
\subfile{./Q2.tex}
\end{document}
```

# 6 Question & Solution Commands

- `\question` — Starts a numbered question with a black dividing rule.
- `\question*{Title}` — Starts a question header with a custom title string.
- `\question#{Points}` — Creates a question header displaying allocated point values.
- `\solution` — Creates a solution header followed by a colon.
- `\solution*{Title}` — Custom solution header with a red dividing rule.
- `alphaparts` / `arabicparts` — Sub-question lists for (a), (b) or 1.1, 1.2 numbering.
- `\answerbox{height}` — Generates an empty framed box for student answers.

# 7 Code Listings with Minted

To include syntax-highlighted code in assignments or solutions, use the `minted` environment (wrap within `\LRE{}` when inside a Persian context):

```python
import numpy as np

def solve_system(A, b):
    """Solve linear system Ax = b and verify residual norm."""
    x = np.linalg.solve(A, b)
    residual = np.linalg.norm(np.dot(A, x) - b)
    return x, residual
```

# 8 Compilation Guide across Environments

Because the template utilizes **XePersian** and TrueType fonts, compilation requires the **XeLaTeX** engine with `-shell-escape` (for `minted` code highlighting). Below is how to run it in various workflows:

## 8.1 1. Overleaf (Recommended if no local TeX distribution is installed)

Overleaf is the easiest option for students without TeX Live or MiKTeX, as it requires zero local installation:

1. Compress the project folder into a `.zip` archive (excluding `LaTeX-build/`).
2. In Overleaf, click **New Project** → **Upload Project** and upload the zip.
3. The included `latexmkrc` automatically sets the compiler to **XeLaTeX** and enables `-shell-escape`. (You can also verify this in Overleaf's left **Menu** → **Compiler** → **XeLaTeX**).
4. Set **Main document** in Menu to `HW.tex` (or `README.tex`) and click **Recompile**.

## 8.2 2. Visual Studio Code + LaTeX Workshop

If you use VS Code with a local TeX distribution (TeX Live or MiKTeX):

1. Install the official **LaTeX Workshop** extension from the VS Code Marketplace.
2. Ensure Python and the `pygments` package are installed (`pip install pygments`).
3. In your VS Code `settings.json`, configure a two-pass XeLaTeX recipe:

```
"latex-workshop.latex.tools": [{
    "name": "xelatex",
    "command": "xelatex",
    "args": ["-synctex=1", "-interaction=nonstopmode", "-output-directory=%OUTDIR%",
             "-file-line-error", "-shell-escape", "%DOC%"]
}],
"latex-workshop.latex.recipes": [{
    "name": "Compile with XeLaTeX",
    "tools": ["xelatex", "xelatex"]
}],
"latex-workshop.latex.outDir": "%DIR%/LaTeX-build"
```

4. Open `HW.tex` or `README.tex` and press `Ctrl + Alt + B` to compile.

## 8.3 3. TeXworks (Standard with MiKTeX & TeX Live)

TeXworks is the lightweight default editor bundled with most Windows TeX installers (including MiKTeX):

1. Open `HW.tex` in TeXworks.
2. Next to the green Play button, select **XeLaTeX** from the dropdown menu (never use pdfLaTeX).
3. To enable code highlighting with `minted`:
   - Navigate to **Edit** → **Preferences** → **Typesetting** tab.
   - Under **Processing tools**, select **XeLaTeX** and click **Edit**.
   - Add a new argument `-shell-escape` and move it before `$synctexoption`. Click OK.
4. Click the green **Play** button to compile. Run it twice on the first build so that minted code and bookmarks populate.

## 8.4 4. Local Distribution Notes (MiKTeX vs. TeX Live)

- If you do not have TeX Live installed, **MiKTeX** is a lightweight alternative. In the MiKTeX Console settings, set *"Install missing packages on-the-fly"* to **Yes**.
- If you prefer not to install any local software at all, use **Overleaf**.